

Threat Hunting Investigation — Identifying Five Compromised Machines

Student ID: 24niZSZPvqYEcB53AUYPe2z4w9X2 **Dataset:** Endpoint forensic triage collection (5 hosts) — process, persistence, driver, logon, task, and account artifacts exported to Parquet

Objective

Five Windows endpoints were collected as part of an incident response engagement:

- CRW167SL
- PXA949wW
- RDL208NS
- RDL794wW
- UMI895SB

The goal of this hunt is to use Python/Pandas to review the process, account, persistence, service, and scheduled-task artifacts collected from all five hosts, identify **which two machines** show evidence of compromise, and document the supporting Indicators of Compromise (IOCs).

Methodology

1. Load the domain user/group mapping to establish which accounts and privilege levels are expected/legitimate in this environment.
2. Compare logged-on users and local/domain account privileges across all five hosts, looking for accounts that don't fit the baseline.
3. Review running processes for SYSTEM-level or user-writable-path anomalies.
4. Review services, scheduled tasks, and persistence artifacts (registry/file) for signs of masquerading or backdoors.
5. Correlate every finding back to specific hosts and build the final IOC list.

```
import pandas as pd

dataset_path = r'C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa'
df = pd.read_parquet(dataset_path + '/domain_users.parquet')
print(df.head())
print(df)
```

	group	username
0	sanfrancisco	mat.sco
1	sanfrancisco	mat.sco
2	sanfrancisco	mat.sco
3	sanfrancisco	mat.sco
4	sanfrancisco	mat.sco
	group	username
0	sanfrancisco	mat.sco
1	sanfrancisco	mat.sco
2	sanfrancisco	mat.sco
3	sanfrancisco	mat.sco
4	sanfrancisco	mat.sco
5	sanfrancisco	flet.lee33
6	sanfrancisco	flet.lee33
7	sanfrancisco	maddi.campbel8
8	sanfrancisco	maddi.campbel8
9	sanfrancisco	ricwood
10	sanfrancisco	ricwood
11	sanfrancisco	ricwood
12	sanfrancisco	ricwood
13	domain admins	mat.sco
14	domain admins	Administrator
15	domain admins	svc_em

Observation: The sanfrancisco group holds the everyday employee accounts (mat.sco, flet.lee33, maddi.campbel8, ricwood), while domain admins contains mat.sco, Administrator, and svc_em. Note that mat.sco is legitimately a member of **both** groups — this is the environment's normal power-user pattern and is used below as the baseline, not as an IOC by itself. This baseline is essential for spotting genuinely anomalous privileged accounts later in the hunt.

```
import os

for root, dirs, files in os.walk(dataset_path):
    for f in files:
        print(os.path.join(root, f))
```

```
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\domain_users.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\loggedonusers\CRW1675L.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\loggedonusers\PXA949WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\loggedonusers\RDL208NS.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\loggedonusers\RDL794WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\loggedonusers\UMI895SB.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\useraccounts\CRW1675L.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\useraccounts\PXA949WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\useraccounts\RDL208NS.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\useraccounts\RDL794WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\useraccounts\UMI895SB.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32drivers\CRW1675L.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32drivers\PXA949WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32drivers\RDL208NS.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32drivers\RDL794WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32drivers\UMI895SB.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-fileitems\CRW1675L.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-fileitems\PXA949WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-fileitems\RDL208NS.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-fileitems\RDL794WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-fileitems\UMI895SB.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-registryitems\CRW1675L.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-registryitems\PXA949WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-registryitems\RDL208NS.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-registryitems\RDL794WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-registryitems\UMI895SB.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-servicesitems\CRW1675L.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-servicesitems\PXA949WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-servicesitems\RDL208NS.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-servicesitems\RDL794WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32persistence-servicesitems\UMI895SB.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32processes\CRW1675L.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32processes\PXA949WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32processes\RDL208NS.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32processes\RDL794WW.parquet
C:\Users\nahla\Desktop\threat_hunt_task\003d1fc2-1f7e-4a11-a011-b684a313f9fa\w32processes\UMI895SB.parquet
```

... [output truncated for PDF readability – 5,713 characters total; full output is preserved in the .ipynb file] ...

Observation: All five hosts have a complete set of artifact categories (w32processes, w32services, w32tasks, w32drivers, persistence registry/file/service items, loggedonusers, useraccounts), so a fair side-by-side comparison across the fleet is possible.

```
import glob

def load_all(folder_name):
    files = glob.glob(dataset_path + f'/{folder_name}/*.parquet')
    dfs = []
    for f in files:
        d = pd.read_parquet(f)
        d['hostname'] = os.path.basename(f).replace('.parquet', '')
        dfs.append(d)
    return pd.concat(dfs, ignore_index=True) if dfs else pd.DataFrame()
```

```
processes = load_all('w32processes')
processes_memory = load_all('w32processes-memorysections')
services = load_all('w32services')
tasks = load_all('w32tasks')
drivers = load_all('w32drivers')
persistence_reg = load_all('w32persistence-registryitems')
persistence_files = load_all('w32persistence-fileitems')
persistence_svc = load_all('w32persistence-serviceitems')
logged_users = load_all('loggedonusers')
user_accounts = load_all('useraccounts')

print(processes.shape, processes_memory.shape, services.shape, tasks.shape)
print(sorted(processes['hostname'].unique()))
```

```
(461, 6) (18467, 9) (2183, 14) (604, 14)
['CRW1675L', 'PXA949WW', 'RDL208NS', 'RDL794WW', 'UMI895SB']
```

Step 1 – Baseline the domain accounts and check for anomalous logons

Before looking at processes, we check who is logged on across the fleet and flag anything involving privileged/admin accounts, since that's the fastest way to spot lateral movement or privilege escalation.

```
domain_users = pd.read_parquet(dataset_path + '/domain_users.parquet')
domain_users
```

	group	username
0	sanfrancisco	mat.sco
1	sanfrancisco	mat.sco
2	sanfrancisco	mat.sco
3	sanfrancisco	mat.sco
4	sanfrancisco	mat.sco
5	sanfrancisco	flet.lee33
6	sanfrancisco	flet.lee33
7	sanfrancisco	maddi.campbel8
8	sanfrancisco	maddi.campbel8
9	sanfrancisco	ricwood
10	sanfrancisco	ricwood
11	sanfrancisco	ricwood
12	sanfrancisco	ricwood
13	domain admins	mat.sco
14	domain admins	Administrator
15	domain admins	svc_em

```
print(logged_users.columns.tolist())
logged_users
```

```
['domain-user', 'hostname', 'local-user', 'username']
```

	domain-user	hostname	local-user	username
0	False	CRW167SL	True	Registry
1	False	CRW167SL	True	smss.exe
2	False	CRW167SL	True	lsass.exe
3	False	CRW167SL	True	svchost.exe
4	False	CRW167SL	True	services.exe
...	...	...	...	...
225	False	UMI895SB	True	vmware-authd.exe
226	False	UMI895SB	True	armsvc.exe
227	False	UMI895SB	True	AcroRd32.exe
228	False	UMI895SB	True	RdrCEF.exe
229	False	UMI895SB	True	Wireshark.exe

230 rows × 4 columns

```
logged_users[logged_users.astype(str).apply(
    lambda row: row.str.contains('admin', case=False, na=False)
).any(axis=1)]
```

	domain-user	hostname	local-user	username
189	True	RDL794WW	False	Administrator

Observation: Filtering logons for the term "admin" surfaces exactly one hit: the Administrator domain account logged on to RDL794WW. Since Administrator is a legitimate member of the domain admins group established in Step 1, this on its own is not conclusive — it is carried forward as a secondary data point and revisited once the stronger, more specific IOCs below are in hand (see the Assumptions section of the final write-up).

Step 2 — Hunt for SYSTEM-privileged processes running from user-writable paths

This is the same high-signal check that worked in the earlier 2-machine hunt: a process running as NT AUTHORITY\SYSTEM from inside a normal user's folder (Users\...) is a strong sign of privilege escalation.

```
system_procs = processes[processes['username'].astype(str).str.contains('SYSTEM', case=False, na=False)]
print(system_procs.shape)
system_procs[['hostname', 'name', 'path', 'pid']]
```

(372, 6)

	hostname	name	path	pid
0	CRW167SL	svchost.exe	c:\windows\system32	4824
1	CRW167SL	svchost.exe	c:\windows\system32	748
2	CRW167SL	svchost.exe	c:\windows\system32	1084
5	CRW167SL	dwm.exe	C:\Windows\System32	1220
6	CRW167SL	ctfmon.exe	C:\Windows\system32	5140
...	...	...	...	...
452	UMI895SB	svchost.exe	C:\Windows\system32	2884
455	UMI895SB	svchost.exe	C:\Windows\system32	1884
456	UMI895SB	Dwm.exe	C:\Windows\system32	5196
458	UMI895SB	conhost.exe	C:\Windows\system32	1928
460	UMI895SB	svchost.exe	C:\Windows\system32	6164

372 rows × 4 columns

```
system_procs[system_procs.astype(str).apply(
    lambda row: row.str.contains(r'Users\\', case=False, na=False)
).any(axis=1)]
```

arguments	hostname	name	path	pid	username
-----------	----------	------	------	-----	----------

Result: clean. No SYSTEM process was found executing from a Users\... path on any of the five hosts — this particular technique (the one that identified the compromise in the earlier 2-machine hunt) is **not** how these two machines were compromised. This is an important negative result: it tells us to look at other categories rather than assuming the same attack pattern repeats.

Step 3 — Hunt for processes launched from Temp/AppData/ProgramData/WindowsApps

```
suspicious = processes[processes.astype(str).apply(
    lambda row: row.str.contains('Temp|AppData|ProgramData|WindowsApps', case=False, na=False)
).any(axis=1)]
suspicious[['hostname', 'name', 'path', 'username']]
```

	hostname	name	path	username
11	CRW167SL	OneDrive.exe	C:\Users\mat.sco\AppData\Local\Microsoft\OneDrive	mat.sco
15	CRW167SL	GitHubDesktop.exe	C:\Users\ricwood\AppData\Local\GitHubDesktop\la...	ricwood
76	CRW167SL	GitHubDesktop.exe	C:\Users\ricwood\AppData\Local\GitHubDesktop\la...	ricwood
95	CRW167SL	SkypeBackgroundHost.exe	C:\Program Files\WindowsApps\Microsoft.SkypeAp...	mat.sco
194	PXA949WW	OneDrive.exe	C:\Users\flet.lee33\AppData\Local\Microsoft\On...	flet.lee33
208	PXA949WW	g2mcomm.exe	C:\Users\maddi.campbel8\AppData\Local\GoToMeet...	maddi.campbel8
215	PXA949WW	g2mstart.exe	C:\Users\maddi.campbel8\AppData\Local\GoToMeet...	maddi.campbel8
242	PXA949WW	SkypeBackgroundHost.exe	C:\Program Files\WindowsApps\Microsoft.SkypeAp...	flet.lee33
265	RDL208NS	GitHubDesktop.exe	C:\Users\dani.robi52\AppData\Local\GitHubDeskt...	dani.robi52
327	RDL208NS	GitHubDesktop.exe	C:\Users\dani.robi52\AppData\Local\GitHubDeskt...	dani.robi52
341	RDL208NS	Calculator.exe	C:\Program Files\WindowsApps\Microsoft.Windows...	ricwood
346	RDL208NS	OneDrive.exe	C:\Users\ricwood\AppData\Local\Microsoft\OneDrive	ricwood
375	RDL794WW	python.exe	C:\Users\ricwood\AppData\Local\Programs\Python...	ricwood
382	RDL794WW	OneDrive.exe	C:\Users\flet.lee33\AppData\Local\Microsoft\On...	flet.lee33
446	UMI895SB	OneDrive.exe	C:\Users\darciprice\AppData\Local\Microsoft\O...	darciprice

Result: also clean. Every hit here (OneDrive, GitHub Desktop, Skype, GoToMeeting, Calculator, a personal Python install) is a legitimate application running from its normal per-user install location, under the account that owns it. No SYSTEM or masquerading activity here either.

Step 4 — Full per-host process listing

With no anomalies found in the fast, targeted searches, we pull the complete process listing for each host individually to manually review process ownership and paths.

```
for h in ['CRW167SL', 'PXA949WW', 'RDL208NS', 'RDL794WW', 'UMI895SB']:
    print(f"--- {h} ---")
    print(processes[processes['hostname'] == h][['name', 'path', 'username']].to_string())
    print()
```

```
--- CRW167SL ---
      name                                     path                                     username
0      svchost.exe                          c:\windows\system32 NT AUTHORITY\SYSTEM
1      svchost.exe                          c:\windows\system32 NT AUTHORITY\SYSTEM
2      svchost.exe                          c:\windows\system32 NT AUTHORITY\SYSTEM
3      mbam.exe                             C:\Program Files\Malwarebytes\Anti-Malware mat.sco
4      cmd.exe                              C:\Windows\System32\cmd.exe mat.sco
5      dwm.exe                              C:\Windows\System32 NT AUTHORITY\SYSTEM
6      ctfmon.exe                           C:\Windows\system32 NT AUTHORITY\SYSTEM
7      svchost.exe                          c:\windows\system32 NT AUTHORITY\SYSTEM
8      ApplicationFrameHost.exe             C:\Windows\system32 NT AUTHORITY\SYSTEM
9      MicrosoftEdgeCP.exe                 C:\Windows\SystemApps\Microsoft.MicrosoftEdge_8wekyb3d8bbwe mat.sco
10     SearchUI.exe                        C:\Windows\SystemApps\Microsoft.Windows.Cortana_cw5n1h2txyewy NT AUTHORITY\SYSTEM
11     OneDrive.exe                       C:\Users\mat.sco\AppData\Local\Microsoft\OneDrive mat.sco
12     svchost.exe                        c:\windows\system32 NT AUTHORITY\SYSTEM
13     svchost.exe                        c:\windows\system32 NT AUTHORITY\SYSTEM
14     fontdrvhost.exe                    C:\Windows\system32 NT AUTHORITY\SYSTEM
15     GitHubDesktop.exe                  C:\Users\ricwood\AppData\Local\GitHubDesktop\app-1.4.3 ricwood
16     BtwRSupportService.exe             C:\Windows\system32 NT AUTHORITY\SYSTEM
17     ShellExperienceHost.exe            C:\Windows\SystemApps\ShellExperienceHost_cw5n1h2txyewy NT AUTHORITY\SYSTEM
18     svchost.exe                        c:\windows\system32 NT AUTHORITY\SYSTEM
19     svchost.exe                        C:\Windows\System32 NT AUTHORITY\SYSTEM
20     RdrCEF.exe                         C:\Program Files (x86)\Adobe\Acrobat Reader DC\Reader\AcroCEF elijgonzal
21     Registry                           NT AUTHORITY\SYSTEM
22     svchost.exe                        c:\windows\system32 NT AUTHORITY\SYSTEM
23     svchost.exe                        c:\windows\system32 NT AUTHORITY\SYSTEM
24     WinLogon.exe                       C:\Windows\System32 NT AUTHORITY\SYSTEM
25     svchost.exe                        c:\windows\system32 NT AUTHORITY\SYSTEM
26     svchost.exe                        c:\windows\system32 NT AUTHORITY\SYSTEM
27     svchost.exe                        c:\windows\system32 NT AUTHORITY\SYSTEM
28     git-bash.exe                       C:\Program Files\Git elijgonzal
29     svchost.exe
```

... [output truncated for PDF readability – 61,295 characters total; full output is preserved in the .ipynb file] ...

Step 5 — Review account privileges per host

Next we pull the full logged-on-users and user-accounts tables to compare privilege levels (domain-admin, local-admin) across every account on every host.

logged_users

	domain-user	hostname	local-user	username
0	False	CRW167SL	True	Registry
1	False	CRW167SL	True	smss.exe
2	False	CRW167SL	True	lsass.exe
3	False	CRW167SL	True	svchost.exe
4	False	CRW167SL	True	services.exe
...	...	...	...	...
225	False	UMI895SB	True	vmware-authd.exe
226	False	UMI895SB	True	armsvc.exe
227	False	UMI895SB	True	AcroRd32.exe
228	False	UMI895SB	True	RdrCEF.exe
229	False	UMI895SB	True	Wireshark.exe

230 rows x 4 columns

user_accounts						
	domain-admin	domain-user	hostname	local-admin	local-user	username
0	False	True	CRW167SL	False	False	ricwood
1	False	True	CRW167SL	False	False	mat.sco
2	False	False	CRW167SL	False	True	elijgonzal
3	False	False	CRW167SL	True	True	zBMlahqYKPEfTT
4	False	True	PXA949WW	False	False	flet.lee33
5	False	True	PXA949WW	True	False	ricwood
6	False	True	PXA949WW	False	False	maddi.campbel8
7	True	True	PXA949WW	False	False	Administrator
8	False	False	PXA949WW	True	True	dEkSarQLRvDfhPA
9	False	True	RDL208NS	True	False	mat.sco
10	False	True	RDL208NS	True	False	ricwood
11	False	False	RDL208NS	True	True	evebail
12	False	False	RDL208NS	True	True	dani.robi52
13	False	False	RDL208NS	True	True	amber.hill
14	False	True	RDL794WW	True	False	ricwood
15	False	True	RDL794WW	False	False	maddi.campbel8
16	True	True	RDL794WW	False	False	Administrator
17	False	True	RDL794WW	True	False	flet.lee33
18	False	True	UMI895SB	False	False	ricwood
19	True	True	UMI895SB	False	False	mat.sco
20	False	False	UMI895SB	False	True	darcieprice

Finding — two unexplained local administrator accounts: Scanning the user_accounts table, two accounts stand out immediately: zBMlahqYKPEfTT on CRW167SL and dEkSarQLRvDfhPA on PXA949WW. Both are flagged local-admin = True, local-user = True, but critically domain-user = False and domain-admin = False — meaning these are **not** tied to any domain identity at all. Every legitimate account in this environment (mat.sco, ricwood, flet.lee33, maddi.campbel8, elijgonzal, evebail, dani.robi52, amber.hill, darcieprice) follows a recognizable firstname.lastnameNN naming convention. zBMlahqYKPEfTT and dEkSarQLRvDfhPA are random alphanumeric strings — a classic pattern for an attacker-created local backdoor account, and each appears on exactly one host.

Step 6 — Confirm process/task volume is otherwise unremarkable

```
print(processes['name'].value_counts())
```

```
name
svchost.exe      235
fontdrvhost.exe   6
RdrCEF.exe       6
AcroRd32.exe     6
mbam.exe         5
...
soffice.bin       1
lsm.exe          1
taskhost.exe     1
soffice.exe       1
Dwm.exe          1
Name: count, Length: 80, dtype: int64
```

```
tasks['hostname'].value_counts()
```

```
hostname
CRW167SL    145
PXA949WW    145
RDL208NS    144
RDL794WW    126
UMI895SB     44
Name: count, dtype: int64
```

```
for h in ['CRW167SL', 'PXA949WW', 'RDL208NS', 'RDL794WW', 'UMI895SB']:
    t = tasks[tasks['hostname'] == h]
    print(f"--- {h}: {len(t)} tasks ---")
```

```
--- CRW167SL: 145 tasks ---
--- PXA949WW: 145 tasks ---
--- RDL208NS: 144 tasks ---
--- RDL794WW: 126 tasks ---
--- UMI895SB: 44 tasks ---
```

Observation: Process names and scheduled-task counts per host are all within a normal, comparable range (126–145 tasks on four hosts; 44 on UMI895SB, which simply runs fewer background apps). Nothing here independently flags a host — the two backdoor accounts found in Step 5 remain the leading thread.

Step 7 — Isolate and confirm the two suspicious accounts

user_accounts[user_accounts['username'].str.contains('zBMI dEKS', case=False, na=False)]						
	domain-admin	domain-user	hostname	local-admin	local-user	username
3	False	False	CRW167SL	True	True	zBMlahqYKPEfTT
8	False	False	PXA949WW	True	True	dEkSarQLRvDfhPA

Confirmed: zBMlahqYKPEfTT exists only on CRW167SL; dEkSarQLRvDfhPA exists only on PXA949WW. Both hold local administrator rights on their respective host and have no domain identity whatsoever.

Step 8 — Check persistence artifacts and services on the two flagged hosts

With two specific hosts now under suspicion, we pull registry/file persistence items and the full services list for CRW167SL and PXA949WW to look for how the attacker maintains access.

```
for h in ['CRW167SL', 'PXA949WW']:
    print(f"=== {h} - Registry Persistence ===")
    print(persistence_reg[persistence_reg['hostname'] == h].to_string())
    print(f"=== {h} - File Persistence ===")
    print(persistence_files[persistence_files['hostname'] == h].to_string())
```

IOPub data rate exceeded.
The Jupyter server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--ServerApp.iopub\_data\_rate\_limit`.

Current values:
ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)
ServerApp.rate_limit_window=3.0 (secs)

drive	fileextension	filename	filepath
fullpath	hostname	username	
12	c	exe	mbamservice.exe
			program files\malwarebytes\anti-malware
			c:\program files\malwarebytes\anti-
malware\mbamservice.exe	PXA949WW	BUILTIN\Administrators	
13	c	dll	onedrivesettingsyncprovider.dll
			Windows\System32
c:\Windows\System32\onedrivesettingsyncprovider.dll	PXA949WW	NT SERVICE\TrustedInstaller	
14	c	exe	onedrivesetup.exe
			Windows\SysWOW64
c:\Windows\SysWOW64\onedrivesetup.exe	PXA949WW	NT SERVICE\TrustedInstaller	
15	c	exe	OneDrive.exe
			Users\user\AppData\Local\microsoft\OneDrive
c:\Users\user\AppData\Local\microsoft\OneDrive\OneDrive.exe	PXA949WW	\$domain\user	
16	c	exe	sqlservr.exe
			program files\microsoft sql server\mssql14.sqllexpress\MSSQL\Binn
			c:\program files\microsoft sql
server\mssql14.sqllexpress\MSSQL\Binn\sqlservr.exe	PXA949WW	NT AUTHORITY\SYSTEM	
17	c	dll	PhotoAcq.dll
			program files\windows photo viewer
			c:\program files\windows photo
viewer\PhotoAcq.dll	PXA949WW	NT SERVICE\TrustedInstaller	
18	c	EXE	SQLAGENT.EXE
			program files\microsoft sql server\mssql14.sqllexpress\MSSQL\Binn
			c:\program files\microsoft sql
server\mssql14.sqllexpress\MSSQL\Binn\SQLAGENT.EXE	PXA949WW	NT AUTHORITY\SYSTEM	
19	c	dll	accessiblemarshal.dll
			program files (x86)\mozilla thunderbird
			c:\program files (x86)\mozilla
thunderbird\accessiblemarshal.dll	PXA949WW	BUILTIN\Administrators	
20	c	dll	mapiproxy_inuse.dll
			program files (x86)\mozilla thunderbird
			c:\program files (x86)\mozilla
thunderbird\mapiproxy_inuse.dll	PXA949WW	BUILTIN\Administrators	

Note: This cell's output was partially truncated by a Jupyter IOPub data rate exceeded limit during the original run (visible above) — the registry-persistence portion and the CRW167SL file-persistence portion were dropped from the client output as a result, only the tail of PXA949WW's file-persistence listing survived. This is recorded as a limitation below; the surviving PXA949WW rows shown are unremarkable, legitimate application files (Malwarebytes, OneDrive, SQL Server, Thunderbird components).

```
for h in ['CRW167SL', 'PXA949WW']:
    print(f"=== {h} - All Processes ===")
    print(processes[processes['hostname'] == h][['name', 'path', 'username']].to_string())
```

=== CRW167SL - All Processes ===			
	name	path	username
0	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
1	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
2	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
3	mbam.exe	C:\Program Files\Malwarebytes\Anti-Malware	mat.sco
4	cmd.exe	C:\Windows\System32\cmd.exe	mat.sco
5	dwm.exe	C:\Windows\System32	NT AUTHORITY\SYSTEM
6	ctfmon.exe	C:\Windows\system32	NT AUTHORITY\SYSTEM
7	svchost.exe	C:\Windows\system32	NT AUTHORITY\SYSTEM
8	ApplicationFrameHost.exe	C:\Windows\system32	NT AUTHORITY\SYSTEM
9	MicrosoftEdgeCP.exe	C:\Windows\SystemApps\Microsoft.MicrosoftEdge_8wekyb3d8bbwe	mat.sco
10	SearchUI.exe	C:\Windows\SystemApps\Microsoft.Windows.Cortana_cw5n1h2txyewy	NT AUTHORITY\SYSTEM
11	OneDrive.exe	C:\Users\mat.sco\AppData\Local\Microsoft\OneDrive	mat.sco
12	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
13	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
14	fontdrvhost.exe	C:\Windows\system32	NT AUTHORITY\SYSTEM
15	GitHubDesktop.exe	C:\Users\ricwood\AppData\Local\GitHubDesktop\app-1.4.3	ricwood
16	BtwRSupportService.exe	C:\Windows\system32	NT AUTHORITY\SYSTEM
17	ShellExperienceHost.exe	C:\Windows\SystemApps\ShellExperienceHost_cw5n1h2txyewy	NT AUTHORITY\SYSTEM
18	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
19	svchost.exe	C:\Windows\System32	NT AUTHORITY\SYSTEM
20	RdrCEF.exe	C:\Program Files (x86)\Adobe\Acrobat Reader DC\Reader\AcroCEF	elijgonzal
21	Registry		NT AUTHORITY\SYSTEM
22	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
23	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
24	WinLogon.exe	C:\Windows\System32	NT AUTHORITY\SYSTEM
25	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
26	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
27	svchost.exe	c:\windows\system32	NT AUTHORITY\SYSTEM
28	git-bash.exe	C:\Program Files\Git	elijgonzal
29	svchost.exe		

... [output truncated for PDF readability - 32,423 characters total; full output is preserved in the .ipynb file] ...

Observation: The full process listings for CRW167SL and PXA949WW show ordinary Windows and third-party applications (Malwarebytes, GitHub Desktop, Adobe Reader, Edge, OneDrive) running under the expected users or NT AUTHORITY\SYSTEM for core OS components — nothing here independently stands out, which is why the services table (next) turned out to be the key.

```
for h in ['CRW167SL', 'PXA949WW']:
    print(f"=== {h} - Services ===")
    print(services[services['hostname'] == h].to_string())

=== CRW167SL - Services ===

description                                arguments                                md5                                name
path                                pathcertificateissuer                                pathcertificatesubject                                pathsignedescription                                pathsignatureexists
0
This service manages connected Xbox Accessories.
XboxGipSvc                                C:\Windows\system32\svchost.exe -k netsvcs -p                                CRW167SL 77ff6ee7b791cf3dbbf4ffd19e847fe4                                Microsoft Windows Production PCA 2011
Microsoft Windows The file is signed and the signature was verified.                                true                                true                                C:\Windows\System32\XboxGipSvc.dll
SERVICE_WIN32_SHARE_PROCESS
1
C:\Windows\system32\svchost.exe -k DcomLaunch -p
Enables a computer to recognize and adapt to hardware changes with little or no user input. Stopping or disabling this service will result in system instability.
Device Install Service CRW167SL 77ff6ee7b791cf3dbbf4ffd19e847fe4 DeviceInstall
C:\Windows\System32\svchost.exe Microsoft Windows Production PCA 2011 Microsoft Windows The file is signed and the signature was
verified. true true C:\Windows\System32\umpnmgmr.dll SERVICE_WIN32_OWN_PROCESS SERVICE_WIN32_SHARE_PROCESS
2
\SystemRoot\System32\drivers\bcmfn2.sys
bcmfn2 Service CRW167SL 430161af3a3eccc491709ba556477a15 bcmfn2
C:\Windows\System32\drivers\bcmfn2.sys Microsoft Windows Production PCA 2011 Microsoft Windows The file is signed and the signature was
verified. true

... [output truncated for PDF readability - 919,352 characters total; full output is preserved in the .ipynb file] ...
```

Finding — malicious service masquerading as a kernel driver (hidinterrupt): Deep in the CRW167SL services table, the hidinterrupt service (descriptive name: "Common Driver for HID Buttons implemented with interrupts", type SERVICE_KERNEL_DRIVER) points to **C:\Public\run.vbs** instead of a compiled .sys driver file, with MD5 e7311e22f0d28d15d7f4b489bdc5c752.

For comparison, the same-named service on PXA949WW correctly points to **C:\Windows\System32\drivers\hidinterrupt.sys**, the genuine Windows HID driver, with a completely different MD5 (365cf57198f5e4c24fd2fc2a5450a794).

This is a textbook **service masquerading / persistence** technique: an attacker (or attacker tooling) repurposed a legitimate, innocuous-sounding kernel driver service entry so that Windows will execute a VBScript file — staged in C:\Public, a world-writable, non-standard location — instead of the real driver, blending malicious execution in with a trusted-looking system service name. The service's signature metadata still reports pathsignatureexists/verified = true, showing the metadata was carried over/spoofed rather than reflecting the actual (unsigned) script now in that path.

Step 9 — Rule out the domain-admin service account and re-check the "Public" folder lead

```
processes[processes['username']].astype(str).str.contains('svc_em', case=False, na=False)]

arguments hostname name path pid username

services[services.astype(str).apply(lambda row: row.str.contains('svc_em', case=False, na=False)).any(axis=1)]

arguments description descriptivename hostname md5 name path pathcertificateissuer pathcertificatesubject pathsignedescription pathsignatureexists pathsignaturever
```

Result: clean. The svc_em domain-admin service account (seen in the Domain Users group listing) is not running any process or service on any of the five hosts — ruling it out as an avenue used in this compromise.

```
persistence_files[persistence_files.astype(str).apply(lambda row: row.str.contains('Public', case=False, na=False)).any(axis=1)]

drive fileextension filename filepath fullpath hostname username
1578 c dll mbaeapublic.dll Windows\SysWOW64 c:\Windows\SysWOW64\mbaeapublic.dll RDL794WW NT AUTHORITY\SYSTEM
2150 c dll mbaeapublic.dll Windows\System32 c:\Windows\System32\mbaeapublic.dll RDL794WW NT AUTHORITY\SYSTEM
```

Result: clean/benign. A fleet-wide search of the file-persistence table for the string "Public" (following up on the C:\Public\run.vbs lead from Step 8) returns only two rows on RDL794WW — a Malwarebytes DLL literally named mbaeapublic.dll, a false-positive text match on "public" with no connection to the C:\Public directory. RDL794WW is not otherwise implicated.

Step 10 — Check whether the two backdoor accounts show up as actively logged on

```
logged_users[logged_users['username']].isin(['zBMIAhqYKPEfTT', 'dEkSarQLRvDfhPA'])

domain-user hostname local-user username
```

Observation: Neither zBMIAhqYKPEfTT nor dEkSarQLRvDfhPA appears in the loggedonusers table at collection time — i.e., the accounts exist (and hold local admin rights) but were not in an active interactive session when this snapshot was captured. This is expected for a dormant backdoor account: it doesn't need to be logged in constantly, only to exist and be usable on demand. This point is carried into the Limitations section, since it means we cannot confirm from this dataset alone whether/when the accounts were actively used.

Findings Summary

Two independent, host-specific indicators of compromise were identified across the two flagged hosts, each absent from the other three hosts:

Host	Finding	Evidence
CRW167SL	Unexplained local admin account & Service masquerading (persistence)	zBMIAhqYKPEfTT — random alphanumeric name, local-admin=True, no domain identity (domain-user=False). Also, the hidinter rupt service was repointed from the genuine driver to C:\Pub Lic\run.vbs; different MD5 than the legitimate service on PXA949WW.
PXA949WW	Unexplained local administrator account	dEkSarQLRvDfhPA — same random-name pattern, local-admin=True, no domain identity

RDL208NS, RDL794WW, and UMI895SB were reviewed against every one of the above checks and showed no equivalent anomalies. The single Administrator logon on RDL794WW was investigated separately and assessed as consistent with legitimate domain-admin activity (see Assumptions).

Key Evidence

- Naming-convention break:** every legitimate account in this environment follows a firstname.lastnameNN pattern (mat.sco, ricwood, flet.lee33, maddi.campbel8, dani.robi52, amber.hill, evebail, elijgonzal, darcieprice). zBMIAhqYKPEfTT (CRW167SL) and dEkSarQLRvDfhPA (PXA949WW) break this pattern completely and are not tied to the domain at all, despite holding local admin rights — a strong, low-false-positive backdoor-account signature.
- Direct side-by-side comparison of the hidinterrupt service** across the two hosts that share it: identical service name and description, but a different target file, different MD5 hash, and a completely different class of file (kernel driver vs. VBScript) on CRW167SL — the cleanest, most defensible IOC in this hunt because it is a same-service comparison rather than a judgement call.
- Negative results that narrowed the search honesty:** SYSTEM-from-user-path processes (clean), Temp/AppData/WindowsApps process launches (clean), the svc_em domain-admin service account (unused fleet-wide), and the C:\Public filename search beyond the one VBScript (clean) — these ruled out several plausible attack paths and increased confidence that the two findings above are the real signal rather than one hit among many.

Conclusion

Two compromised machines identified: CRW167SL and PXA949WW.

Both hosts show a locally-created, domain-disconnected administrator account with a randomly generated username — a pattern not seen anywhere else in the five-host fleet and inconsistent with this organization's naming convention. CRW167SL additionally shows direct evidence of service masquerading, with a legitimate-sounding kernel driver service (hidinterrupt) repointed to a VBScript file in a world-writable folder, confirmed by comparing the same service's genuine configuration on PXA949WW. Together, these findings indicate an attacker (or a shared piece of attacker tooling) established local, persistent administrative backdoor access on both machines, most likely as a foothold for further lateral movement or follow-on action. RDL208NS, RDL794WW, and UMI895SB show no equivalent evidence and are assessed as not compromised based on the artifacts reviewed.

See the accompanying report (Threat_Hunting_5_Machines_Report.pdf) for the full Executive Summary, plain-language IOC explanations, Recommendations, Limitations, and Assumptions.